User Interface Design

Design Models

- The design phase of software development transforms the customer requirements as described in the SRS documents into design forms.
- Designing a model is an important phase and is a multi-process that represents the data structure, program structure, interface characteristics, and procedural details.
- **There are four types of design elements / models:**
    1. Data Design Element / Model
    2. User Interface Design Element / Model
    3. Architectural Design Element / Model
    4. Component Level Design Element / Model

User Interface Design Model

- User interface is the front-end application view to which the user interacts with the software.
- It determines how commands are given to the computer or the program and how data is displayed on the screen.

The software becomes more popular if its user interface is:

1. Attractive
2. Simple to use
3. Responsive in short time
4. Clear to understand
5. Consistent on all interfacing screens

Types of User Interface

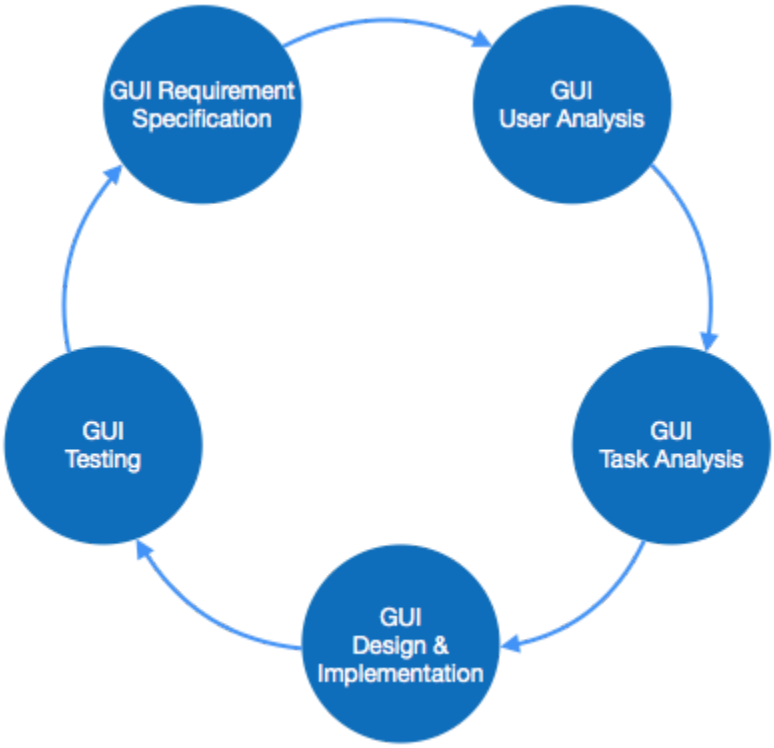
Type 1: Text-Based User Interface OR Command Line Interface

- The text-based interface primarily uses a keyboard for handling data.
- Command Line Interface provides command prompt or coding tools where the user types commands to the system.
- The user needs to remember the syntax of commands and their use.
- Command Line Interface is used by technical people or programmers.

Type 2: Graphical User Interface

- Graphical User Interface provides the simple interactive interface to interact with the system
- GUI can be a combination of both hardware and software.
- Graphical user interfaces are easy to learn as compared to the command-line interface.
- GUI provides multiple windows to the user simultaneously to interact with the system.

User Interface Design Process Steps (Cycle):



User Interface Design Process

- **GUI Requirement Gathering:**
  - The designer analyzes all functional and non-functional requirements discussed with customers.
- **User Analysis:**
  - Designer studies who will use the software GUI. The target audience matters as design depends on user knowledge level.
- **Task Analysis:**
  - The designer analyzes tasks, subtasks, and flow of the system.
- **GUI Design & Implementation:**
  - The designer creates the actual GUI using tools like Wavemaker, Visual Studio, etc.
- **Testing:**
  - GUI testing is done by internal teams, organizations, and stakeholders.

User Interface Design Principles

- **User Familiarity:**
  - Interfaces should use user-oriented terms and concepts.
  - *Example:* Desktop, Document, Folder, Rename, Power (Windows OS).
- **Consistency:**
  - Commands and menus should have the same format and parameters.
  - *Example:* Same fonts, sizes, colors.
- **Minimal Surprise:**
  - The system should not behave unexpectedly.
  - Users get frustrated otherwise. Design based on user expectations.
  - *Example:* File → New, Open, Save, Print in MS Word.
- **Recoverability:**
  - Users make mistakes; the system should allow recovery.
  - *Example:* Undo, Rename.
- **User Diversity:**
  - Interfaces should support different types of users.
  - *Example:* Larger fonts, privacy settings.
- **User Guidance:**
  - Interfaces should give feedback on errors and provide help.
  - *Example:* User manual, Forgot Password.

User Interface Design Issues

- **Response Time:**
  - Time between request and system response.
  - If too long → user frustration.
- **Error Handling:**
  - Poor error messages may lead to rejection of the product.
- **Help Facilities:**
  - Users need help when they cannot find information.
- **Application Accessibility:**
  - Determines whether the application is easy to use or not.
  - Guidelines should be provided during interaction.

Architectural Design Model

The architectural design model functions as a structural blueprint for the system, prioritizing fundamental design decisions at an early stage of the software development lifecycle.

Architectural Design Styles describe:

- A collection of hardware and software components responsible for executing required functions *(e.g., Databases, Modules, Frameworks).*
- A set of connectors facilitating coordination and communication between various components.
- Specific conditions for the successful integration of components into a unified system.
- Semantic models utilized to comprehend the overall properties of the system.

Importance of Software Architecture

1. **Security:** Safeguards the system through the implementation of encryption and various security mechanisms.
2. **Performance:** Optimizes the system to handle requests and responses with high efficiency.
3. **Maintainability:** Ensures the ease of modifying or replacing individual software components.
4. **Safety:** Minimizes operational risk by isolating critical system functionalities.
5. **Availability:** Effectively manages errors to guarantee continuous system operation.

Decisions of Architectural Design

- Determining how the system is distributed across the network infrastructure.
- Selecting the appropriate approach for structuring the entire system.
- Identifying the most suitable architectural styles for the project requirements.
- Defining the methods for documenting the software architecture.
- Strategizing the division of the system into distinct modules.
- Establishing the control strategy to be employed within the architecture.
- Determining the techniques for analyzing the architectural design.

Architectural Views

- Architectural views are utilized to represent the entire system architecture in a manner that is meaningful to stakeholders.

Types of Views:

1. **Use Case View:** Represents the user perspective based on requirements using use case diagrams.
2. **Design View:** Illustrates the design structure, key features, entities, and their attributes.
3. **Process View:** Describes system communication, behavior, and synchronization processes.
4. **Implementation View:** Focuses on source code organization, developer roles, and system integration.
5. **Deployment View:** Displays the physical system environment where the application resides and executes.

Taxonomy of Architectural Styles

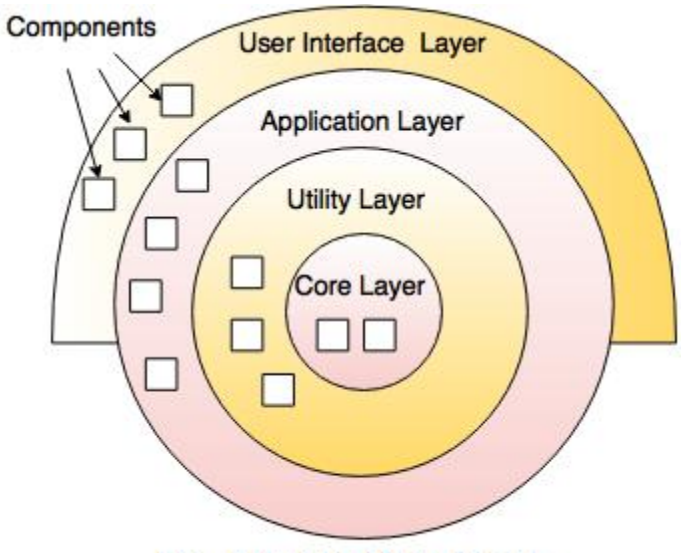
Architectural styles establish the fundamental structure and organization of system components.

Types:

1. Data Centered Architecture
2. Data Flow Architecture
3. Object Oriented Architecture
4. Layered Architecture

Layered Architecture

- In this model, data transitions through multiple layers for sequential processing.
- Every layer is designed to perform a specific and well-defined function.

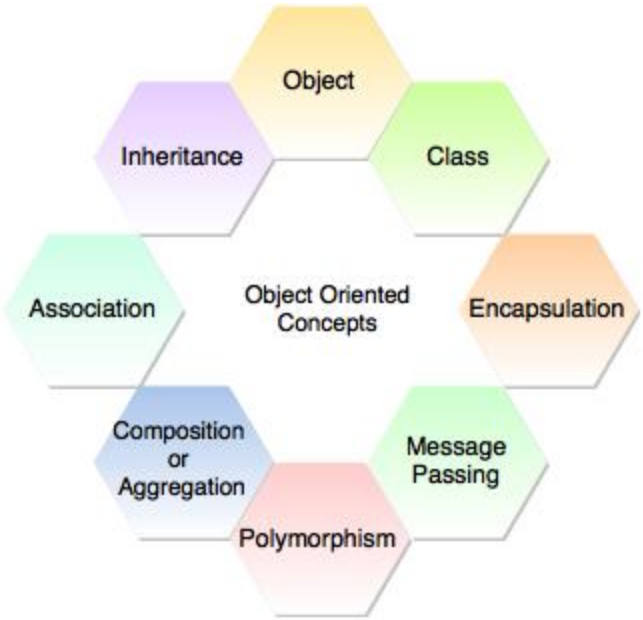


Layers:

- **Outer Layer:** Manages all user interface operations.
- **Intermediate Layers:** Provide utility functions and application-level services.
- **Inner Layer:** Facilitates direct interaction with the Operating System.

Object Oriented Architecture

- Objects serve as the fundamental building blocks of the software system.



Concepts:

1. **Object:** An instance of a class *(e.g., Student)*.
2. **Class:** Defines the specific attributes and methods for objects.
3. **Encapsulation:** The process of binding related elements together.
4. **Abstraction:** The removal of non-essential details to simplify complexity.
5. **Inheritance:** Creating a new class derived from an existing one.
6. **Polymorphism:** Allows a single object to take on multiple forms.
7. **Message Passing:** Enables coordination and communication between objects.

Data Flow Architecture

- This architecture transforms input data into output results through a sequence of processing components.

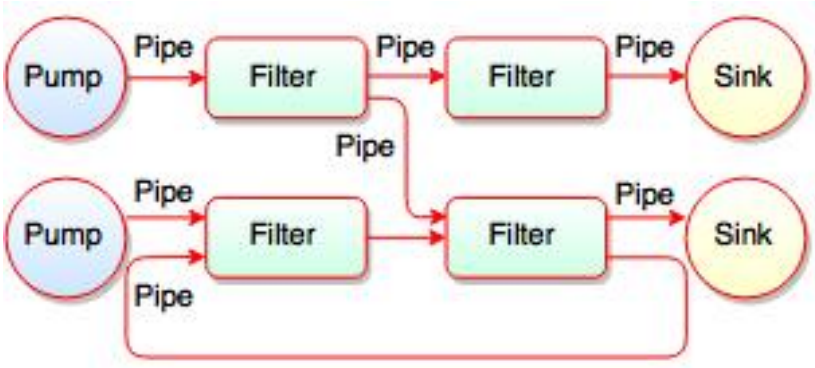


Fig. Pipes and Filters

Components:

- **Pipe:** Acts as a conduit for transferring data between system components.
- **Filter:** Processes incoming data streams to produce meaningful output.

Advantages:

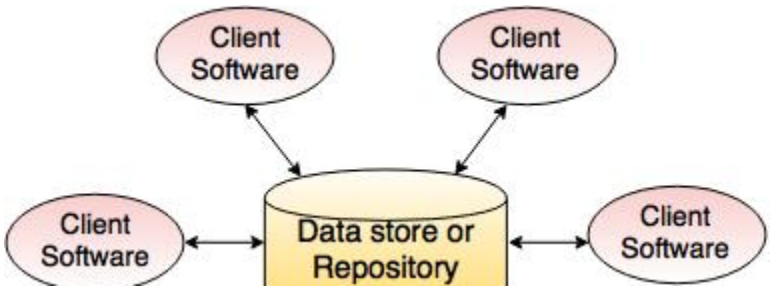
- Supports the concurrent execution of various processes.

Disadvantages:

- Typically provides less opportunity for direct user interaction.

Data Centered Architecture

- Data is maintained in a centralized location for efficient storage and management.
- The architecture allows multiple users to access the data simultaneously.



Features:

- Enables users to add, update, delete, and modify stored data records.

Advantages:

1. Operates independently of individual client configurations.
2. Facilitates the seamless addition of new clients to the system.
3. Simplifies the modification of system components.

Disadvantages:

1. Risk of data duplication within the central repository.
2. Centralized changes may impact the functionality of all connected clients.